

ACTIVIDAD 4

K-Means y reducción de dimensiones con PCA

Agrupamiento de la base Iris

Gael García

1. Objetivo de la actividad

En esta actividad se aplicó el algoritmo K-Means a la base Iris. Primero se trabajó con las cuatro variables numéricas originales y se buscó un número adecuado de grupos mediante el gráfico del codo y el indicador Silhouette. Después se redujeron las cuatro variables a dos componentes con PCA, se repitió K-Means y se compararon los agrupamientos obtenidos.

2. Base de datos

La base contiene 150 observaciones de flores y cinco columnas. Para K-Means se utilizaron sepal.length, sepal.width, petal.length y petal.width. La columna variety se conservó como referencia, pero no se incluyó en el algoritmo porque contiene categorías.

sepal.length	sepal.width	petal.length	petal.width	variety
5.1	3.5	1.4	0.2	Setosa
4.9	3.0	1.4	0.2	Setosa
4.7	3.2	1.3	0.2	Setosa
4.6	3.1	1.5	0.2	Setosa
5.0	3.6	1.4	0.2	Setosa

3. Carga y preparación de los datos

3.1 Librerías y carga del archivo

```
import os
import matplotlib.pyplot as mp
import pandas as pd
import seaborn as sb

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

dataset = pd.read_csv("iris.csv")
dataset
```

El archivo se cargó correctamente y se obtuvo un DataFrame de 150 filas por 5 columnas.

3.2 Selección de variables numéricas

```
fields = dataset.iloc[:, 0:4].values  
print(fields)
```

Se seleccionaron las cuatro variables numéricas de Iris y se excluyó variety. Con esto se obtuvo una matriz de 150 observaciones por 4 variables.

```
[3]: fields = dataset.iloc[:, 0:4].values  
     print(fields)
```

```
[[5.1 3.5 1.4 0.2]  
 [4.9 3.  1.4 0.2]  
 [4.7 3.2 1.3 0.2]  
 [4.6 3.1 1.5 0.2]  
 [5.  3.6 1.4 0.2]  
 [5.4 3.9 1.7 0.4]  
 [4.6 3.4 1.4 0.3]  
 [5.  3.4 1.5 0.2]  
 [4.4 2.9 1.4 0.2]  
 [4.9 3.1 1.5 0.1]  
 [5.4 3.7 1.5 0.2]  
 [4.8 3.4 1.6 0.2]  
 [4.8 3.  1.4 0.1]  
 [4.3 3.  1.1 0.1]  
 [5.8 4.  1.2 0.2]  
 [5.7 4.4 1.5 0.4]  
 [5.4 3.9 1.3 0.4]  
 [5.1 3.5 1.4 0.3]]
```

Selección de las variables numéricas de la base Iris.

4. K-Means con los datos originales

4.1 Cálculo de WCSS

```
wcss = []  
  
for index in range(1, 11):  
    kmeans = KMeans(  
        n_clusters=index,  
        init="k-means++",  
        random_state=2  
    )  
    kmeans.fit(fields)  
    wcss.append(kmeans.inertia_)  
  
wcss
```

El WCSS disminuyó conforme aumentó el número de clusters. La caída fue especialmente fuerte entre 1, 2 y 3 grupos.

4.2 Gráfico del codo

```
sb.set()
mp.plot(range(1, 11), wcss)
mp.title("K-Means Clustering")
mp.xlabel("Número de clusters")
mp.ylabel("WCSS")
mp.show()
```

```
[6]: # Gráfico del codo
sb.set()
```

```
mp.plot(range(1, 11), wcss)
mp.title("K-Means Clustering")
mp.xlabel("Número de clusters")
mp.ylabel("WCSS")
mp.show()
```

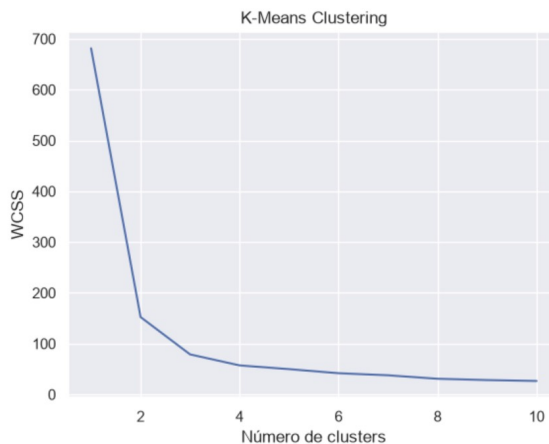


Gráfico del codo con los datos originales.

En el gráfico se observa un cambio claro de pendiente alrededor de $k = 3$. Por este criterio, tres clusters es una opción razonable.

4.3 Indicador Silhouette

```
for j in range(2, 11):
    kmeans = KMeans(n_clusters=j, random_state=42)
    kmeans.fit_predict(fields)
    score = silhouette_score(fields, kmeans.labels_, metric="euclidean")
    print("Score Silhouette; k =", j, ":", score)
```

```
[7]: for j in range(2, 11):
      kmeans = KMeans(
          n_clusters=j,
          random_state=42
      )

      kmeans.fit_predict(fields)

      score = silhouette_score(
          fields,
          kmeans.labels_,
          metric="euclidean"
      )

      print("Score Silhouette; k =", j, ":", score)
```

```
Score Silhouette; k = 2 : 0.6810461692117464
Score Silhouette; k = 3 : 0.5511916046195922
Score Silhouette; k = 4 : 0.4976433179321928
Score Silhouette; k = 5 : 0.4930804067193526
Score Silhouette; k = 6 : 0.36784649847122486
Score Silhouette; k = 7 : 0.3542978877198852
Score Silhouette; k = 8 : 0.34467972180562
Score Silhouette; k = 9 : 0.315588785338977
Score Silhouette; k = 10 : 0.30141437453251435
```

Resultados del indicador Silhouette con los datos originales.

El valor más alto de Silhouette fue para $k = 2$, con aproximadamente 0.6810. Para $k = 3$ se obtuvo aproximadamente 0.5512. Por lo tanto, el codo sugiere 3 grupos, mientras que Silhouette favorece 2.

4.4 Aplicación de K-Means con 3 clusters

```
kmeans = KMeans(  
    n_clusters=3,  
    init="k-means++",  
    random_state=42  
)  
  
cluster_values = kmeans.fit_predict(fields)  
print(cluster_values)
```

```
[8]: kmeans = KMeans(  
      n_clusters=3,  
      init="k-means++",  
      random_state=42  
    )  
  
cluster_values = kmeans.fit_predict(fields)  
  
print(cluster_values)  
  
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1  
 1 1 1 1 1 1 1 1 1 1 1 1 1 0 2 0 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2  
 2 2 2 0 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 0 2 0 0 0 2 0 0 0  
 0 0 2 2 0 0 0 0 2 0 2 0 2 0 2 0 0 0 0 0 2 0 0 0 0 2 0 0 0 2 0 0 2 0  
 0 2]
```

Asignación de cada observación a uno de los tres clusters.

Para continuar la comparación se utilizaron 3 clusters, de acuerdo con el gráfico del codo. Las etiquetas 0, 1 y 2 solo identifican grupos y no equivalen directamente a las especies de Iris.

4.5 Guardar el grupo junto a la base

```
dataset_original = dataset.copy()  
dataset_original["grupo_kmeans"] = cluster_values  
  
dataset_original
```

```
- - -  
[9]: dataset_original = dataset.copy()  
      dataset_original["grupo_kmeans"] = cluster_values  
  
      dataset_original  
  
[9]:
```

	sepal.length	sepal.width	petal.length	petal.width	variety	grupo_kmeans
0	5.1	3.5	1.4	0.2	Setosa	1
1	4.9	3.0	1.4	0.2	Setosa	1
2	4.7	3.2	1.3	0.2	Setosa	1
3	4.6	3.1	1.5	0.2	Setosa	1
4	5.0	3.6	1.4	0.2	Setosa	1
...	...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Virginica	0
146	6.3	2.5	5.0	1.9	Virginica	2
147	6.5	3.0	5.2	2.0	Virginica	0
148	6.2	3.4	5.4	2.3	Virginica	0
149	5.9	3.0	5.1	1.8	Virginica	2

150 rows x 6 columns

Base original con la columna del grupo asignado por K-Means.

5. Reducción de dimensiones con PCA

5.1 Transformación de 4 variables a 2 componentes

```
fields2 = dataset.iloc[:, 0:4].values

from sklearn import decomposition

pca = decomposition.PCA(n_components=2)
pca.fit(fields2)
fields2 = pca.transform(fields2)
fields2
```

```
[10]: # Datos para PCA
fields2 = dataset.iloc[:, 0:4].values

from sklearn import decomposition

pca = decomposition.PCA(n_components=2)
pca.fit(fields2)

fields2 = pca.transform(fields2)

fields2
```

```
[10]: array([[ -2.68412563,  0.31939725],
        [ -2.71414169, -0.17700123],
        [ -2.88899057, -0.14494943],
        [ -2.74534286, -0.31829898],
        [ -2.72871654,  0.32675451],
        [ -2.28085963,  0.74133045],
        [ -2.82053775, -0.08946138],
        [ -2.62614497,  0.16338496],
        [ -2.88638273, -0.57831175],
        [ -2.6727558 , -0.11377425],
        [ -2.50694709,  0.6450689 ],
        [ -2.61275523,  0.01472994],
        [ -2.78610927, -0.235112  ],
        [ -3.22380374, -0.51139459],
        [ -2.64475039,  1.17876464],
        [ -2.38603903,  1.33806233],
        [ -2.62352788,  0.81067951],
        [ -2.64829671,  0.31184914],
```

Matriz de 150 observaciones reducida a dos componentes mediante PCA.

PCA redujo las cuatro variables numéricas originales a dos componentes. Esto permite visualizar posteriormente los grupos en una gráfica de dos dimensiones.

5.2 WCSS después de PCA

```
wcss2 = []

for index in range(1, 11):
    kmeans = KMeans(
        n_clusters=index,
        init="k-means++",
        random_state=2
    )
    kmeans.fit(fields2)
    wcss2.append(kmeans.inertia_)

wcss2
```

Los valores de WCSS volvieron a disminuir conforme aumentó el número de grupos. El comportamiento general fue similar al obtenido con los datos originales.

5.3 Gráfico del codo después de PCA

```
sb.set()
mp.plot(range(1, 11), wcss2)
mp.title("K-Means Clustering después de PCA")
mp.xlabel("Número de clusters")
mp.ylabel("WCSS")
mp.show()
```

```
[12]: # Gráfico del codo después de PCA
sb.set()
```

```
mp.plot(range(1, 11), wcss2)
mp.title("K-Means Clustering después de PCA")
mp.xlabel("Número de clusters")
mp.ylabel("WCSS")
mp.show()
```

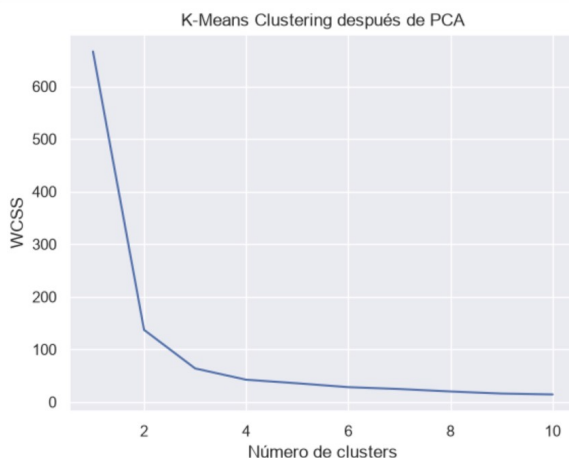


Gráfico del codo después de reducir los datos con PCA.

El segundo gráfico del codo también muestra un cambio de pendiente alrededor de $k = 3$. Así, el criterio del codo se mantuvo prácticamente igual después de PCA.

5.4 Silhouette después de PCA

```
for j in range(2, 11):
    kmeans = KMeans(n_clusters=j, random_state=42)
    kmeans.fit_predict(fields2)
    score = silhouette_score(fields2, kmeans.labels_, metric="euclidean")
    print("Score Silhouette; k =", j, ":", score)
```

```
[13]: # Silhouette después de PCA
```

```
for j in range(2, 11):
    kmeans = KMeans(
        n_clusters=j,
        random_state=42
    )

    kmeans.fit_predict(fields2)

    score = silhouette_score(
        fields2,
        kmeans.labels_,
        metric="euclidean"
    )

    print("Score Silhouette; k =", j, ":", score)
```

```
Score Silhouette; k = 2 : 0.705670322510211
Score Silhouette; k = 3 : 0.5976764219547994
Score Silhouette; k = 4 : 0.5577409232246783
Score Silhouette; k = 5 : 0.5100407194899199
Score Silhouette; k = 6 : 0.40319738737442684
Score Silhouette; k = 7 : 0.38682639977136096
Score Silhouette; k = 8 : 0.44076033249654034
Score Silhouette; k = 9 : 0.40201438763663666
Score Silhouette; k = 10 : 0.4158865801753239
```

Resultados de Silhouette después de PCA.

Después de PCA, Silhouette volvió a favorecer $k = 2$. El valor máximo fue aproximadamente 0.7057, mientras que para $k = 3$ fue aproximadamente 0.5977.

6. K-Means después de PCA

6.1 Aplicación con 3 clusters

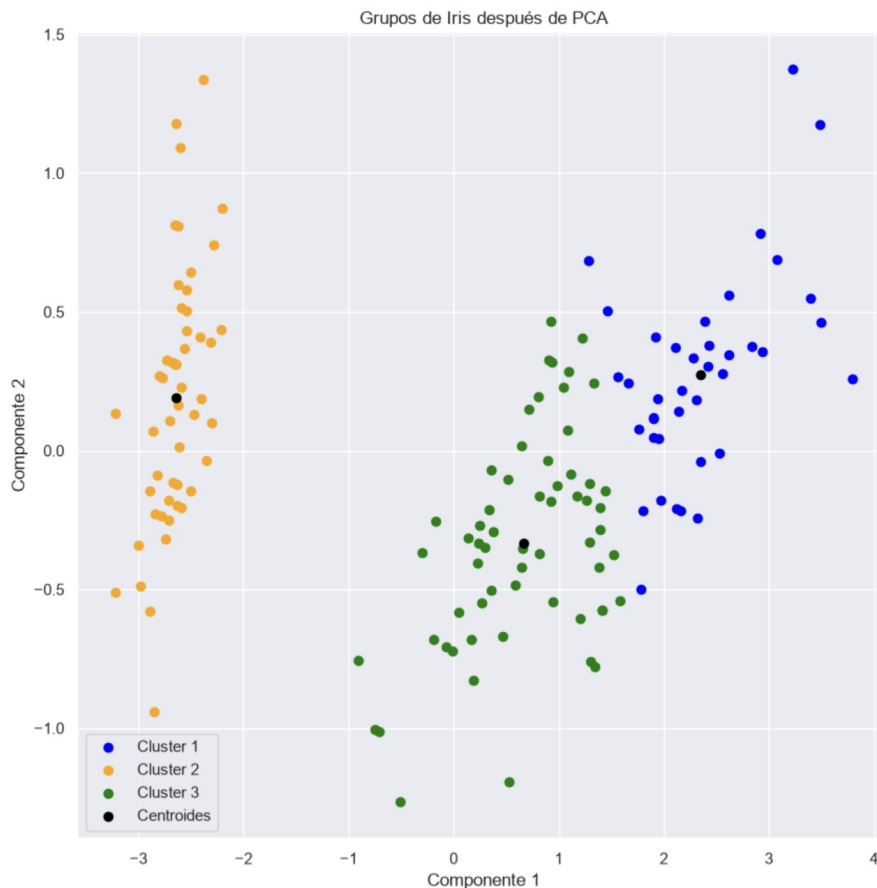
```
kmeans_pca = KMeans(  
    n_clusters=3,  
    init="k-means++",  
    random_state=42  
)  
  
cluster_values_pca = kmeans_pca.fit_predict(fields2)  
print(cluster_values_pca)
```

```
[14]: # K-Means después de PCA con 3 clusters  
kmeans_pca = KMeans(  
    n_clusters=3,  
    init="k-means++",  
    random_state=42  
)  
  
cluster_values_pca = kmeans_pca.fit_predict(fields2)  
  
print(cluster_values_pca)  
  
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1  
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1  
 2 2 2 0 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2  
 0 0 2 2 0 0 0 0 2 2 0 2 0 0 2 2 0 0 0 0 2 0 0 0 0 2 0 0 0 2 0 0  
 0 2]
```

Asignación de clusters utilizando las dos componentes del PCA.

6.2 Gráfica de los grupos y centroides

```
mp.figure(figsize=(10, 10))  
  
mp.scatter(fields2[cluster_values_pca == 0, 0], fields2[cluster_values_pca == 0, 1], c="blue",  
    label="Cluster 1")  
mp.scatter(fields2[cluster_values_pca == 1, 0], fields2[cluster_values_pca == 1, 1], c="orange",  
    label="Cluster 2")  
mp.scatter(fields2[cluster_values_pca == 2, 0], fields2[cluster_values_pca == 2, 1], c="green",  
    label="Cluster 3")  
mp.scatter(kmeans_pca.cluster_centers[:, 0], kmeans_pca.cluster_centers[:, 1], c="black",  
    label="Centroides")  
  
mp.title("Grupos de Iris después de PCA")  
mp.xlabel("Componente 1")  
mp.ylabel("Componente 2")  
mp.legend()  
mp.show()
```



Clusters obtenidos en las dos componentes de PCA. Los puntos negros representan los centroides.

La gráfica permite distinguir visualmente los tres grupos. Uno aparece claramente separado hacia la izquierda, mientras que los otros dos se distribuyen a la derecha con una zona de mayor cercanía entre ellos.

7. Comparación antes y después de PCA

```
comparacion = pd.DataFrame({
    "Grupo_original": cluster_values,
    "Grupo_PCA": cluster_values_pca
})
comparacion
```

La comparación directa permite revisar si cada observación conservó su grupo después de reducir las variables con PCA.

7.1 Tabla cruzada de los agrupamientos

```
pd.crosstab(
    comparacion["Grupo_original"],
    comparacion["Grupo_PCA"]
)
```

```
[17]: # Tabla de comparación
pd.crosstab(
    comparacion["Grupo_original"],
    comparacion["Grupo_PCA"]
)

[17]:
```

Grupo_PCA	0	1	2
Grupo_original			
0	39	0	0
1	0	50	0
2	0	0	61

Tabla cruzada de grupos originales contra grupos después de PCA.

La tabla quedó completamente diagonal: 39 observaciones permanecieron en el grupo 0, 50 en el grupo 1 y 61 en el grupo 2. En total, $39 + 50 + 61 = 150$ observaciones conservaron exactamente la misma asignación.

8. Conclusiones

El análisis mostró que el gráfico del codo sugiere aproximadamente 3 clusters tanto con los datos originales como con los datos reducidos mediante PCA. Por otra parte, el indicador Silhouette alcanzó su valor más alto con 2 clusters en ambos casos.

Para comparar los agrupamientos se utilizó K-Means con 3 clusters. Antes y después de PCA, las 150 observaciones quedaron asignadas exactamente a los mismos grupos: 39 en el grupo 0, 50 en el grupo 1 y 61 en el grupo 2.

La ventaja práctica de PCA fue reducir cuatro variables a dos componentes y permitir una visualización directa de los grupos sin modificar las asignaciones obtenidas por K-Means en este ejercicio. Así, la representación se vuelve más sencilla y conserva el patrón de agrupamiento observado con los datos originales.

Resumen de resultados

Criterio	Datos originales	Después de PCA
Gráfico del codo	≈ 3 clusters	≈ 3 clusters
Silhouette máximo	$k = 2$ (0.6810)	$k = 2$ (0.7057)